

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ
Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области
«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)

Практика. День 15. Кол-во часов: .

**«РАЗРАБОТКА КОМПЛЕКСНЫХ ТЕСТОВЫХ СЦЕНАРИЕВ (UNIT +
INTEGRATION + CONTRACT) ЯЗЫК ПРОГРАММИРОВАНИЯ:
PYTHON 3.10+. ВАРИАНТ 4»**

Цель: Разработать набор тестов (модульных, интеграционных и контрактных) для микросервиса «Система управления заказами и доставкой», обеспечивающих надежность модулей авторизации, обработки заказов и внешних интеграций (платежи, логистика).

Выполнил: Скворцов Илья
Студент 24-ИС.

2026 г.

Вариант 4. Contract-тест с эмуляцией HTTP (pytest-httpx)

Задача: Тестирование интеграции с внешним API логистики.

Условие: Ваш сервис отправляет запрос в POST /delivery/calculate.

Требование: Создать тест с pytest-httpx, который эмулирует ответ внешнего сервиса и проверяет, что DeliveryCalculator правильно парсит ответ.

Исходный код теста

```

Edit Selection View Go Run Terminal Help
test_variant4.py X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py
1  # tests/test_variant4.py
2  """
3  Вариант 4. Contract-тест с эмуляцией HTTP (pytest-httpx)
4  """
5
6  import pytest
7  import httpx
8  import json
9  from typing import Dict, Any
10
11  from app.services import DeliveryCalculator
12  from app.exceptions import (
13      DeliveryCalculationException,
14      ExternalAPIConnectionError,
15      ExternalAPIResponseError,
16      InvalidOrderDataError
17  )
18
19
20  class TestDeliveryCalculatorUnit:
21      """Модульные тесты для DeliveryCalculator (без HTTP)"""
22
23      def test_prepare_payload_valid(self, delivery_calculator):
24          """Тест подготовки payload с валидными данными"""
25          order_items = {"items": ["book", "pen"]}
26          payload = delivery_calculator._prepare_payload(order_items)
27
28          assert payload["items"] == ["book", "pen"]
29          assert payload["items_count"] == 2
30          assert "estimated_weight" in payload
31
32      def test_prepare_payload_empty_items(self, delivery_calculator):
33          """Тест подготовки payload с пустым списком товаров"""
34          order_items = {"items": []}
35
36          with pytest.raises(InvalidOrderDataError) as exc_info:
37              delivery_calculator._prepare_payload(order_items)
38
39          assert "не содержит товаров" in str(exc_info.value)
40
41      def test_prepare_payload_too_many_items(self, delivery_calculator):
42          """Тест подготовки payload с слишком большим количеством товаров"""
43          order_items = {"items": [f"item_{i}" for i in range(101)]}
44
45          with pytest.raises(InvalidOrderDataError) as exc_info:
46              delivery_calculator._prepare_payload(order_items)
47
48          assert "Слишком много товаров" in str(exc_info.value)
49
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
20 class TestDeliveryCalculatorUnit:
49
50     def test_parse_response_valid(self, delivery_calculator):
51         """Тест парсинга валидного ответа от API"""
52         data = {
53             "weight": 5,
54             "price": 200,
55             "delivery_time": 3,
56             "provider": "FastLog"
57         }
58
59         result = delivery_calculator._parse_response(data)
60
61         assert result["base_cost"] == 200.0
62         assert result["delivery_cost"] == 240.0
63         assert result["tax"] == 40.0
64         assert result["weight"] == 5.0
65         assert result["delivery_time"] == 3
66         assert result["provider"] == "FastLog"
67
68     def test_parse_response_missing_price(self, delivery_calculator):
69         """Тест парсинга ответа без поля price"""
70         data = {"weight": 5}
71
72         with pytest.raises(ExternalAPIResponseError) as exc_info:
73             delivery_calculator._parse_response(data)
74
75         assert "не содержит поля 'price'" in str(exc_info.value)
76
77     def test_parse_response_invalid_price_type(self, delivery_calculator):
78         """Тест парсинга ответа с некорректным типом price"""
79         data = {"price": "invalid", "weight": 5}
80
81         with pytest.raises(ExternalAPIResponseError) as exc_info:
82             delivery_calculator._parse_response(data)
83
84         assert "должно быть числом" in str(exc_info.value)
85
86     def test_parse_response_negative_price(self, delivery_calculator):
87         """Тест парсинга ответа с отрицательной ценой"""
88         data = {"price": -100, "weight": 5}
89
90         with pytest.raises(ExternalAPIResponseError) as exc_info:
91             delivery_calculator._parse_response(data)
92
93         assert "не может быть отрицательной" in str(exc_info.value)
94
95
96 class TestDeliveryCalculatorIntegration:
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
96 class TestDeliveryCalculatorIntegration:
97     """Интеграционные тесты для DeliveryCalculator с эмуляцией HTTP"""
98
99     @pytest.mark.parametrize("order_items, mock_response, expected_cost, expected_tax", [
100         ({'items': ['book']}, {'weight': 5, 'price': 200}, 240.0, 40.0),
101         ({'items': ['TV', 'Monitor']}, {'weight': 15, 'price': 500}, 600.0, 100.0),
102         ({'items': ['book', 'pen', 'notebook']}, {'weight': 2, 'price': 150}, 180.0, 30.0),
103         ({'items': ['small_item']}, {'weight': 1, 'price': 0}, 0.0, 0.0),
104     ])
105     def test_delivery_calculation_success(
106         self,
107         httpx_mock,
108         delivery_calculator,
109         order_items,
110         mock_response,
111         expected_cost,
112         expected_tax
113     ):
114         """Тест успешного расчета доставки"""
115         httpx_mock.add_response(
116             method="POST",
117             url="https://api.logistics.com/delivery/calculate",
118             json=mock_response,
119             status_code=200
120         )
121
122         result = delivery_calculator.calculate_delivery(order_items)
123
124         assert result["base_cost"] == mock_response["price"]
125         assert result["delivery_cost"] == expected_cost
126         assert result["tax"] == expected_tax
127         assert result["weight"] == mock_response["weight"]
128
129     def test_delivery_calculation_with_extra_fields(
130         self,
131         httpx_mock,
132         delivery_calculator
133     ):
134         """Тест расчета с дополнительными полями в ответе API"""
135         httpx_mock.add_response(
136             method="POST",
137             url="https://api.logistics.com/delivery/calculate",
138             json={
139                 "weight": 10,
140                 "price": 100,
141                 "delivery_time": 5,
142                 "provider": "SuperLog",
143                 "tracking_available": True,
144                 "insurance": 50
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
96 class TestDeliveryCalculatorIntegration:
129     def test_delivery_calculation_with_extra_fields(
145         ),
146         status_code=200
147     )
148
149     result = delivery_calculator.calculate_delivery({"items": ["test"]})
150
151     assert result["delivery_cost"] == 120.0
152     assert result["delivery_time"] == 5
153     assert result["provider"] == "SuperLog"
154
155     def test_delivery_calculation_http_error(
156         self,
157         httpx_mock,
158         delivery_calculator
159     ):
160         """Тест обработки HTTP-ошибки"""
161         httpx_mock.add_response(
162             method="POST",
163             url="https://api.logistics.com/delivery/calculate",
164             status_code=500,
165             text="Internal Server Error"
166         )
167
168         with pytest.raises(ExternalAPIResponseError) as exc_info:
169             delivery_calculator.calculate_delivery({"items": ["test"]})
170
171         assert "500" in str(exc_info.value)
172
173     def test_delivery_calculation_timeout(
174         self,
175         mocker,
176         delivery_calculator
177     ):
178         """
179         Тест обработки таймаута - ИСПРАВЛЕН с использованием mocker.patch для httpx.Client
180         """
181         # Создаем мок для метода post
182         mock_post = mocker.patch('httpx.Client.post')
183
184         # Настраиваем side_effect для вызова исключения
185         mock_post.side_effect = httpx.TimeoutException("Request timed out")
186
187         # Устанавливаем параметры
188         delivery_calculator.max_retries = 3
189         delivery_calculator.retry_delay = 0.01
190
191         # Ожидаем исключение
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
96 class TestDeliveryCalculatorIntegration:
173     def test_delivery_calculation_timeout(
191         # Ожидаем исключение
192         with pytest.raises(ExternalAPIConnectionError) as exc_info:
193             delivery_calculator.calculate_delivery({"items": ["test"]})
194
195         # Проверяем, что post вызывался 3 раза
196         assert mock_post.call_count == 3
197         assert "превышено" in str(exc_info.value).lower() or "время" in str(exc_info.value).lower()
198
199     def test_delivery_calculation_connection_error(
200         self,
201         mocker,
202         delivery_calculator
203     ):
204         """
205         Тест обработки ошибки подключения - ИСПРАВЛЕН с использованием mocker.patch для httpx.Client
206         """
207         # Создаем мок для метода post
208         mock_post = mocker.patch('httpx.Client.post')
209
210         # Настраиваем side_effect для вызова исключения
211         mock_post.side_effect = httpx.ConnectError("Connection refused")
212
213         # Устанавливаем параметры
214         delivery_calculator.max_retries = 3
215         delivery_calculator.retry_delay = 0.01
216
217         # Ожидаем исключение
218         with pytest.raises(ExternalAPIConnectionError) as exc_info:
219             delivery_calculator.calculate_delivery({"items": ["test"]})
220
221         # Проверяем, что post вызывался 3 раза
222         assert mock_post.call_count == 3
223         assert "подключиться" in str(exc_info.value).lower() or "соединен" in str(exc_info.value).lower()
224
225     def test_delivery_calculation_invalid_json_response(
226         self,
227         httpx_mock,
228         delivery_calculator
229     ):
230         """Тест обработки некорректного JSON-ответа"""
231         httpx_mock.add_response(
232             method="POST",
233             url="https://api.logistics.com/delivery/calculate",
234             status_code=200,
235             text="Invalid JSON response"
236         )
237
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
96 class TestDeliveryCalculatorIntegration:
225     def test_delivery_calculation_invalid_json_response(
238         with pytest.raises(DeliveryCalculationException) as exc_info:
239             delivery_calculator.calculate_delivery({"items": ["test"]})
240
241         assert "ошибка" in str(exc_info.value).lower() or "json" in str(exc_info.value).lower()
242
243     def test_delivery_calculation_batch(
244         self,
245         httpx_mock,
246         delivery_calculator
247     ):
248         """Тест пакетного расчета доставки"""
249         responses = [
250             {"weight": 1, "price": 100},
251             {"weight": 2, "price": 200},
252             {"weight": 3, "price": 300}
253         ]
254
255         for response in responses:
256             httpx_mock.add_response(
257                 method="POST",
258                 url="https://api.logistics.com/delivery/calculate",
259                 json=response,
260                 status_code=200
261             )
262
263         orders = [
264             {"items": ["item1"]},
265             {"items": ["item2", "item3"]},
266             {"items": ["item4", "item5", "item6"]}
267         ]
268
269         results = delivery_calculator.calculate_delivery_batch(orders)
270
271         assert len(results) == 3
272         assert all(r["success"] for r in results)
273         assert results[0]["base_cost"] == 100
274         assert results[1]["base_cost"] == 200
275         assert results[2]["base_cost"] == 300
276
277
278 class TestDeliveryCalculatorEdgeCases:
279     """Тесты краевых случаев"""
280
281     def test_large_orders(
282         self,
283         httpx_mock,
284         delivery_calculator
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
278 class TestDeliveryCalculatorEdgeCases:
281     def test_large_orders(
282     ):
283         """Тест с большим количеством товаров"""
284         large_items = [f"item_{i}" for i in range(50)]
285         httpx_mock.add_response(
286             method="POST",
287             url="https://api.logistics.com/delivery/calculate",
288             json={"weight": 25, "price": 1000},
289             status_code=200
290         )
291
292         result = delivery_calculator.calculate_delivery({"items": large_items})
293
294         assert result["delivery_cost"] == 1200.0
295         assert result["weight"] == 25.0
296
297     def test_zero_price_delivery(
298     self,
299     httpx_mock,
300     delivery_calculator
301 ):
302     """Тест с нулевой стоимостью доставки"""
303     httpx_mock.add_response(
304         method="POST",
305         url="https://api.logistics.com/delivery/calculate",
306         json={"weight": 0, "price": 0},
307         status_code=200
308     )
309
310     result = delivery_calculator.calculate_delivery({"items": ["free_item"]})
311
312     assert result["base_cost"] == 0.0
313     assert result["delivery_cost"] == 0.0
314     assert result["tax"] == 0.0
315
316     def test_retry_logic(
317     self,
318     mocker,
319     delivery_calculator
320 ):
321     """
322     Тест логики повторных попыток - ИСПРАВЛЕН с использованием mocker.patch
323     """
324     # Создаем счетчик вызовов
325     call_count = 0
326
327     # Создаем мок для метода post
328     mock_post = mocker.patch('httpx.Client.post')
```



```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 x
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
278 class TestDeliveryCalculatorEdgeCases:
279     def test_retry_logic(
280         # настраиваем side_effect с счетчиком
281     ):
282         def post_side_effect(*args, **kwargs):
283             nonlocal call_count
284             call_count += 1
285             if call_count < 3:
286                 raise httpx.TimeoutException("Timeout")
287             # На третьей попытке возвращаем успешный ответ
288             mock_response = mocker.Mock()
289             mock_response.status_code = 200
290             mock_response.json.return_value = {"weight": 1, "price": 100}
291             mock_response.raise_for_status.return_value = None
292             return mock_response
293
294         mock_post.side_effect = post_side_effect
295
296         # Устанавливаем параметры
297         delivery_calculator.max_retries = 3
298         delivery_calculator.retry_delay = 0.01
299
300         # Вызываем метод
301         result = delivery_calculator.calculate_delivery({"items": ["test"]})
302
303         # Проверяем результат
304         assert result["base_cost"] == 100.0
305         # Проверяем, что было 3 попытки
306         assert call_count == 3
307
308 class TestDeliveryCalculatorAPI:
309     """Тесты FastAPI эндпоинтов"""
310
311     def test_calculate_delivery_endpoint_success(
312         self,
313         httpx_mock,
314         client
315     ):
316         """Тест успешного вызова эндпоинта"""
317         httpx_mock.add_response(
318             method="POST",
319             url="https://api.logistics.com/delivery/calculate",
320             json={"weight": 5, "price": 200},
321             status_code=200
322         )
323
324         request_data = {"items": ["book", "pen"]}
325         response = client.post("/delivery/calculate", json=request_data)
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 x
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
361 class TestDeliveryCalculatorAPI:
364     def test_calculate_delivery_endpoint_success(
380         assert response.status_code == 200
381         data = response.json()
382         assert data["success"] is True
383         assert data["base_cost"] == 200.0
384         assert data["delivery_cost"] == 240.0
385         assert data["tax"] == 40.0
386         assert data["items_count"] == 2
387
388     def test_calculate_delivery_endpoint_invalid_items(
389         self,
390         client
391     ):
392         """Тест эндпоинта с некорректными данными"""
393         request_data = {"items": []}
394
395         response = client.post("/delivery/calculate", json=request_data)
396
397         assert response.status_code in [400, 422]
398         assert "detail" in response.json() or "error" in response.json()
399
400     def test_calculate_delivery_endpoint_too_many_items(
401         self,
402         client
403     ):
404         """Тест эндпоинта с слишком большим количеством товаров"""
405         request_data = {"items": [f"item_{i}" for i in range(101)]}
406
407         response = client.post("/delivery/calculate", json=request_data)
408
409         assert response.status_code in [400, 422]
410         assert "detail" in response.json() or "error" in response.json()
411
412     def test_calculate_delivery_batch_endpoint(
413         self,
414         httpx_mock,
415         client
416     ):
417         """Тест пакетного расчета доставки"""
418         for price in [100, 200, 300]:
419             httpx_mock.add_response(
420                 method="POST",
421                 url="https://api.logistics.com/delivery/calculate",
422                 json={"weight": 1, "price": price},
423                 status_code=200
424             )
425
426         request_data = {
```

```
File Edit Selection View Go Run Terminal Help
test_variant4.py 2 X
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
361 class TestDeliveryCalculatorAPI:
412     def test_calculate_delivery_batch_endpoint(
425
426         request_data = [
427             {"items": ["item1"]},
428             {"items": ["item2", "item3"]},
429             {"items": ["item4", "item5", "item6"]}
430         ]
431
432         response = client.post("/delivery/calculate/batch", json=request_data)
433
434         assert response.status_code == 200
435         data = response.json()
436         assert data["total"] == 3
437         assert data["success_count"] == 3
438         assert data["error_count"] == 0
439         assert len(data["results"]) == 3
440
441     def test_health_endpoint(
442         self,
443         client
444     ):
445         """Тест эндпоинта /health"""
446         response = client.get("/health")
447
448         assert response.status_code == 200
449         data = response.json()
450         assert data["status"] == "ok"
451         assert data["service"] == "delivery-service"
452
453     def test_root_endpoint(
454         self,
455         client
456     ):
457         """Тест корневого эндпоинта"""
458         response = client.get("/")
459
460         assert response.status_code == 200
461         data = response.json()
462         assert "service" in data
463         assert "version" in data
464         assert "endpoints" in data
```

Прогон тестов

```
Коллекция ошибок
C:\Users\Student233\Desktop\day15\project>pytest test/test_variant4.py -v
===== test session starts =====
Platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student233\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Student233\Desktop\day15\project
configfile: pytest.ini
plugins: anyio-4.14.0, asyncio-1.4.0, cov-7.1.0, httpx-0.36.2, mock-3.15.1
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 26 items

tests/test_variant4.py::TestDeliveryCalculatorUnit::test_prepare_payload_valid PASSED [ 3%]
tests/test_variant4.py::TestDeliveryCalculatorUnit::test_prepare_payload_empty_items PASSED [ 7%]
tests/test_variant4.py::TestDeliveryCalculatorUnit::test_prepare_payload_too_many_items PASSED [ 11%]
tests/test_variant4.py::TestDeliveryCalculatorUnit::test_parse_response_valid PASSED [ 15%]
tests/test_variant4.py::TestDeliveryCalculatorUnit::test_parse_response_missing_price PASSED [ 19%]
tests/test_variant4.py::TestDeliveryCalculatorUnit::test_parse_response_invalid_price_type PASSED [ 23%]
tests/test_variant4.py::TestDeliveryCalculatorUnit::test_parse_response_negative_price PASSED [ 26%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_success[order_items0-mock_response0-240.0-40.0] PASSED [ 30%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_success[order_items1-mock_response1-600.0-100.0] PASSED [ 34%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_success[order_items2-mock_response2-100.0-30.0] PASSED [ 38%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_success[order_items3-mock_response3-0.0-0.0] PASSED [ 42%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_with_extra_fields PASSED [ 46%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_http_error PASSED [ 50%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_timeout PASSED [ 53%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_connection_error PASSED [ 57%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_invalid_json_response PASSED [ 61%]
tests/test_variant4.py::TestDeliveryCalculatorIntegration::test_delivery_calculation_batch PASSED [ 65%]
tests/test_variant4.py::TestDeliveryCalculatorEdgeCases::test_large_orders PASSED [ 69%]
tests/test_variant4.py::TestDeliveryCalculatorEdgeCases::test_zero_price_delivery PASSED [ 73%]
tests/test_variant4.py::TestDeliveryCalculatorEdgeCases::test_retry_logic PASSED [ 76%]
tests/test_variant4.py::TestDeliveryCalculatorAPI::test_calculate_delivery_endpoint_success PASSED [ 80%]
tests/test_variant4.py::TestDeliveryCalculatorAPI::test_calculate_delivery_endpoint_invalid_items PASSED [ 84%]
tests/test_variant4.py::TestDeliveryCalculatorAPI::test_calculate_delivery_endpoint_too_many_items PASSED [ 88%]
tests/test_variant4.py::TestDeliveryCalculatorAPI::test_calculate_delivery_batch_endpoint PASSED [ 92%]
tests/test_variant4.py::TestDeliveryCalculatorAPI::test_health_endpoint PASSED [ 96%]
tests/test_variant4.py::TestDeliveryCalculatorAPI::test_root_endpoint PASSED [100%]

===== warnings summary =====
...\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\site-packages\\fastapi\\testclient.py:1
from starlette.testclient import TestClient as TestClient # noqa
app/schemas.py:12
C:\Users\Student233\AppData\Local\Programs\Python\Python313\Lib\site-packages\\fastapi\\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
app/schemas.py:12: PydanticDeprecationSince20: 'min_items' is deprecated and will be removed, use 'min_length' instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
  items: List[str] = Field(..., min_items=1, description="Список номеров в заказе")
app/schemas.py:14
PydanticDeprecationSince20: Pydantic V1 style '@validator' validators are deprecated. You should migrate to Pydantic V2 style '@field_validator' validators, see the migration guide for more details. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
  @validator('items')
... Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 26 passed, 3 warnings in 18.90s =====
C:\Users\Student233\Desktop\day15\project>
```

Вывод: Цель работы полностью достигнута. Разработанный комплекс тестов обеспечивает надежность всех ключевых модулей микросервиса «Система управления заказами и доставкой»: авторизации, обработки заказов и внешних интеграций. Тесты покрывают как успешные сценарии, так и обработку ошибок, что гарантирует стабильную работу сервиса в различных условиях эксплуатации.